Simulado de Prova 2 – Programação Orientada a Objetos
Nível: entre médio e difícil · Duração sugerida: 3h30

PRONTUÁRIO: INDICAR NO ZIP

✓ A prova é individual e sem consulta. ✓ Não é permitido utilizar qualquer código implementado por você anteriormente, exceto o fornecido nos Anexos. ✓ Não coloque seu nome no projeto.	✓ O projeto deve ser nomeado PRONTUARIO_P2. ✓ Não envie apenas as classes, mas todo o projeto. ✓ Atribui-se nota zero à prova em desacordo com os itens acima.

Considere o contexto a seguir:

Pizzaria, delivery e muita fome! A **PizzaTech** decidiu abandonar a planilha e você foi contratado(a) para criar o sistema de pedidos. Os produtos do cardápio são carregados de um arquivo e organizados por categoria (pizzas, bebidas e sobremesas). Cada pedido passa por várias etapas desde que o cliente começa a montá-lo até a entrega: ele é pago, preparado, despachado e entregue, mas pode ser cancelado em algumas dessas etapas e, dependendo de quando isso acontece, o cliente recebe um reembolso. Ações fora de hora, como entregar um pedido que ainda não foi preparado, não podem ser aceitas. A Tabela 1 representa os estados do pedido e o que acontece quando cada evento ocorre.

Tabela 1 – Eventos e transições de estado de um pedido

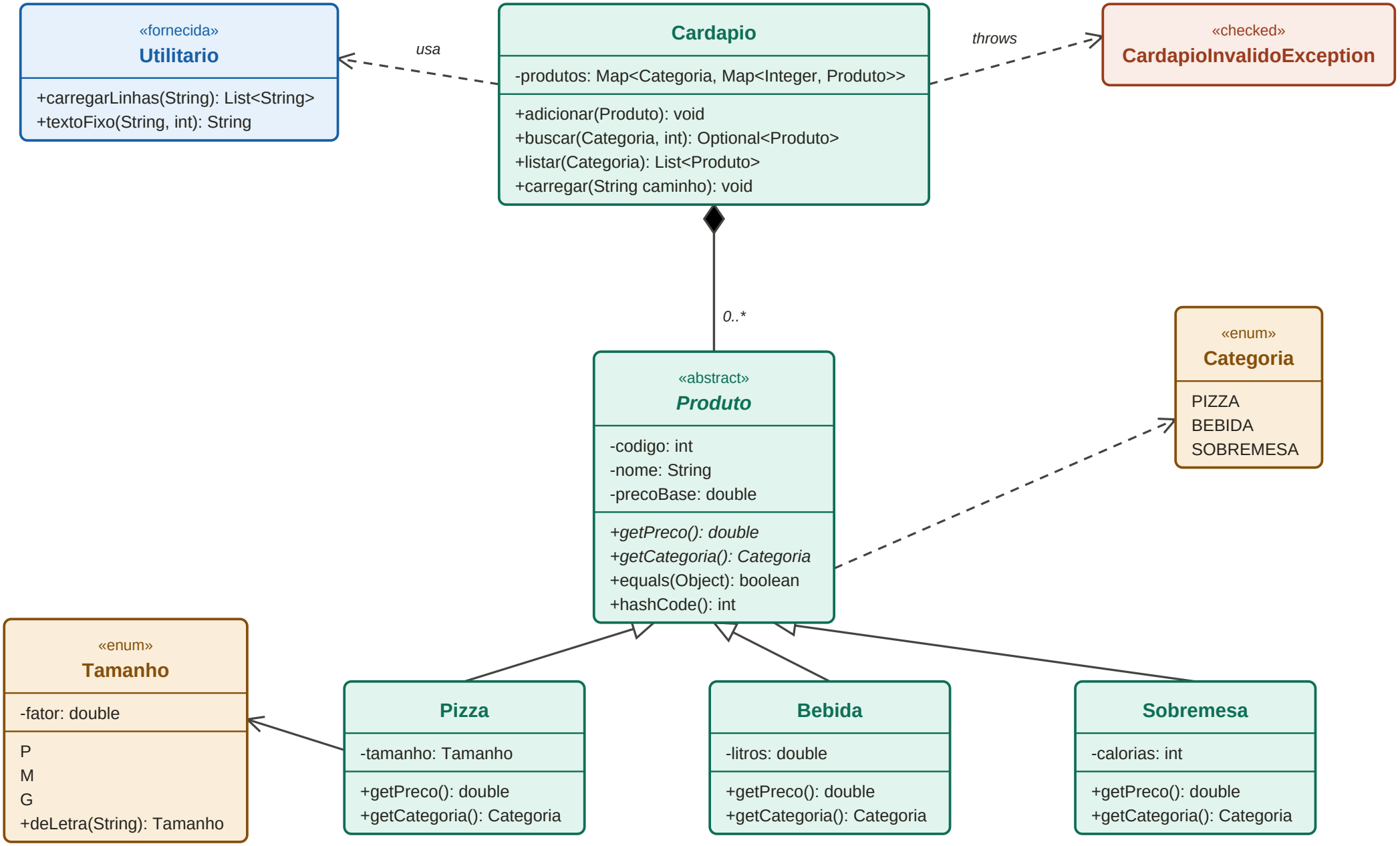
	adicionarItem	**pagar**	**preparar**	**despachar**	**entregar**	**cancelar**
Aberto	adiciona o item	Pago (ERRO se vazio)	ERRO	ERRO	ERRO	Cancelado (reembolso 0%)
Pago	ERRO	ERRO	EmPreparo	ERRO	ERRO	Cancelado (reembolso 100%)
EmPreparo	ERRO	ERRO	ERRO	EmRota	ERRO	Cancelado (reembolso 50%)
EmRota	ERRO	ERRO	ERRO	ERRO	Entregue	ERRO
Entregue	---	---	---	---	---	---
Cancelado	---	---	---	---	---	---

ERRO: o evento não é permitido naquele estado e deve provocar uma *RegraDeNegocioException*. Nos estados finais (Entregue e Cancelado) nenhum evento é aceito.

Como você aprendeu herança e polimorfismo, aplicou o padrão de projeto *State* para encapsular cada etapa do pedido em um objeto, evitando uma série de *ifs* e *elses* pouco flexíveis. Além disso, seguindo a boa prática de modelar o domínio com classes de objetos e suas relações, você desenhou o modelo dos produtos e do cardápio. O design completo é ilustrado nas **Figuras 1 e 2**, nas páginas seguintes. Use-as com atenção: os modificadores de visibilidade, os tipos genéricos e as relações fazem parte da especificação.

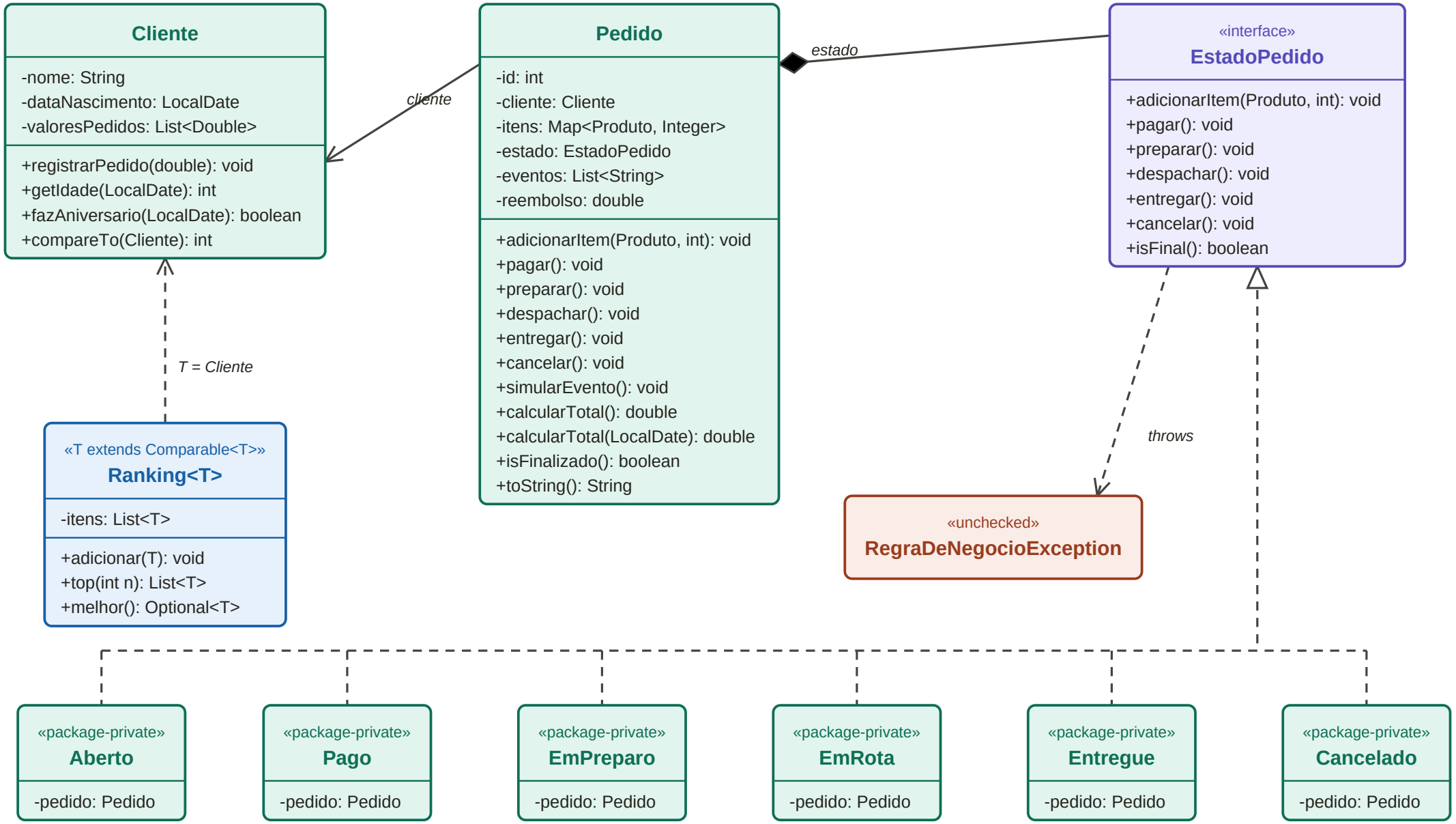
Para dar mais realismo, o cardápio é lido do arquivo *cardapio.csv* (Anexo B), com os campos de cada produto separados por ";", por meio da classe *Utilitario* fornecida (Anexo A). Os resultados devem seguir os formatos dos Quadros 1 a 4, ao final.

Figura 1 - Modelo de dominio: produtos, cardapio e utilitarios



Italico = abstrato. Linha tracejada com seta = dependencia. Losango preto = composicao. Triangulo vazio = heranca. Construtores, getters e setters omitidos por clareza.

Figura 2 - Pedido adotando o padrao State, Cliente e Ranking generico



Todos os estados implementam os 7 metodos de EstadoPedido (omitidos nas caixas).

Triangulo vazio tracejado = implementa interface. Losango preto = composicao. Seta tracejada = dependencia. Construtores, getters e setters omitidos por clareza.

Leia atentamente e execute as atividades a seguir para a implementação do exercício em **Java 21**. Siga a ordem dos itens: cada um depende dos anteriores. Você pode criar outros métodos, principalmente privados, para melhor organizar sua aplicação. Para a atribuição da nota será levada em conta não apenas a funcionalidade, mas a qualidade, adequação e pertinência de cada solução. Bom senso faz parte da prova.

Dica: *obj.getClass().getSimpleName()* retorna o nome da classe do objeto. Para formatar valores monetários no padrão brasileiro, use *String.format(Locale.forLanguageTag("pt-BR"), "%.2f", valor)*.

#	Descrição	Pontos
1	Crie o projeto com quatro pacotes: **main**, **modelo**, **pedido** e **util**. Em *main*, crie a classe *Principal*, que conterá o método *main*. Em *util*, inclua a classe *Utilitario* fornecida (Anexo A) sem alterá-la. Em *modelo* ficam produtos, cardápio, cliente e enums; em *pedido*, o padrão State.	0,25pt
2	Crie os enums *Categoria* (PIZZA, BEBIDA, SOBREMESA) e *Tamanho* (P, M, G). Em *Tamanho*, inclua o atributo *fator* (0,8; 1,0 e 1,3, respectivamente) e o método de classe *deLetra(String)*, que converte “p”, “M”, “g” etc. (sem diferenciar maiúsculas de minúsculas) e lança *IllegalArgumentException* com mensagem explicativa para qualquer outra entrada.	0,25pt
3	Crie a classe abstrata *Produto* (código, nome e preço-base) com os métodos abstratos *getCategoria()* e *getPreco()*, e as subclasses *Pizza* (tamanho; preço = preço-base × fator), *Bebida* (litros; preço = preço-base × litros) e *Sobremesa* (calorias; preço = preço-base). Valide os construtores (nome nulo ou vazio, preço-base ou litros menores ou iguais a zero) lançando exceção adequada. Sobrescreva *toString()* nas subclasses, como no Quadro 1. Use os modificadores de acesso mais adequados ao encapsulamento.	1,0pt
4	Em *Produto*, implemente *equals()* e *hashCode()* considerando apenas o código. Comente em uma linha por que isso é necessário para que o *Pedido* use *Map<Produto, Integer>* corretamente.	0,25pt
5	Crie as exceções *RegraDeNegocioException* (**não verificada**, pacote *pedido*) e *CardapioInvalidoException* (**verificada**, pacote *modelo*). Ambas devem ter um construtor com *String message* e outro com *String message, Throwable cause*.	0,5pt
6	Crie a classe *Cardapio*, que armazena os produtos em *Map<Categoria, Map<Integer, Produto>>* (use *EnumMap* para o mapa externo). Implemente: *adicionar(Produto)*, que lança exceção adequada se o código já existir na categoria; *buscar(Categoria, int)*, que retorna *Optional<Produto>*; e *listar(Categoria)*, que retorna os produtos ordenados por nome e uma lista vazia (nunca *null*) se não houver nenhum. A busca por código deve ser direta, sem percorrer coleções.	0,5pt
7	Implemente *carregar(String caminho)* em *Cardapio*, declarando *throws CardapioInvalidoException*. Use *Utilitario.carregarLinhas()* e, para cada linha não vazia, extraia os cinco campos com *split()* (tipo; código; nome; preço-base; extra). O tipo define a subclasse a ser criada e o campo extra é o tamanho (pizza), os litros (bebida) ou as calorias (sobremesa). Qualquer problema (arquivo inexistente, número de campos errado, número ou tipo inválidos) deve ser convertido em *CardapioInvalidoException*, com mensagem contendo o número da linha (ex.: “Linha 4: preço inválido”) e a causa original preservada. Isole exceções de terceiros: nenhuma exceção do Java deve sair do método.	1,0pt
8	Crie a classe *Cliente* (nome, data de nascimento como *LocalDate* e lista de valores dos pedidos entregues), que implementa *Comparable<Cliente>*. Implemente *registrarPedido(double)*; *getIdade(LocalDate hoje)* usando *Period*; e *fazAniversario(LocalDate hoje)* (mesmo dia e mês). Recuse datas de nascimento futuras. O *compareTo()* compara primeiro o maior valor de pedido de cada cliente; em caso de empate, o segundo maior, e assim sucessivamente. Se todos os valores comparados forem iguais, o cliente com mais pedidos é o maior. Persistindo o empate, usa-se a ordem alfabética do nome. Sobrescreva *toString()* no formato “Cliente = Maria | Maior pedido = R$ 215,40” (use R$ 0,00 se não houver pedidos).	1,0pt
9	Crie a classe genérica *Ranking<T extends Comparable<T>>*, com os métodos *adicionar(T)*, *top(int n)* e *melhor()*. O *top* devolve os *n* maiores elementos em ordem decrescente (se *n* for maior que o total, devolve todos; se *n* for menor ou igual a zero, lança exceção adequada) e não pode expor a lista interna. O *melhor* devolve *Optional<T>*. Explique em um comentário o que aconteceria se o limite *extends Comparable<T>* fosse removido.	0,5pt

#	Descrição	Pontos
10	Crie a interface *EstadoPedido* com os métodos da Figura 2 e a classe *Pedido*. Todo pedido começa no estado *Aberto*, tem um id sequencial gerado automaticamente (atributo de classe), um cliente, os itens em um *Map<Produto, Integer>* (use *merge()* para somar quantidades do mesmo produto) e um valor de reembolso. Implemente *adicionarItem()*, *pagar()*, *preparar()*, *despachar()*, *entregar()* e *cancelar()* delegando polimorficamente a ação ao estado atual, e *isFinalizado()*. **Não utilize if, else ou switch sobre o estado.** Exponha os itens por um getter que impeça alterações externas.	1,0pt
11	Implemente em *Pedido* o método *calcularTotal(LocalDate hoje)*, que soma preço × quantidade de cada item e aplica 10% de desconto se hoje for aniversário do cliente (arredonde para duas casas decimais). Crie também a sobrecarga *calcularTotal()*, que utiliza a data atual.	0,5pt
12	Na classe *Pedido*, implemente o registro de eventos, com o formato do Quadro 2. O método *mudarEstado(novoEstado, descricaoDoEvento)*, utilizado pelos estados, altera o estado e registra o evento. O método *registrarRejeicao(evento, mensagem)* registra um evento recusado sem alterar o estado. Em ambos, informe o timestamp atual (*LocalDateTime* e *DateTimeFormatter*), o id do pedido, o evento, a transição de estado (que pode ser o mesmo) e o total. Você não deve imprimir nenhum evento fora da classe *Principal*. Sobrescreva *toString()* para retornar todos os eventos, um por linha, **sem utilizar o operador +** para concatenar Strings.	0,75pt
13	Implemente as seis classes concretas de *EstadoPedido* (Aberto, Pago, EmPreparo, EmRota, Entregue e Cancelado), seguindo a Tabela 1. Receba o *Pedido* no construtor e adote para as classes e seus construtores um modificador de acesso que permita visibilidade apenas dentro do pacote *pedido*. Em *Aberto*, pagar um pedido sem itens também é erro. Ao cancelar, guarde no pedido o valor do reembolso conforme a Tabela 1. Ao entregar, registre o valor total no cliente (*registrarPedido*). As mensagens das exceções devem indicar o evento e o estado atual.	1,0pt
14	Implemente *simularEvento()* em *Pedido*, que sorteia uma das cinco ações (pagar, preparar, despachar, entregar ou cancelar) e a invoca. Se a ação for recusada com *RegraDeNegocioException*, a exceção deve ser capturada e registrada com *registrarRejeicao()*, sem interromper o programa. Use uma única instância de *Random* compartilhada pela classe, e não uma nova a cada chamada.	0,5pt
15	Crie a classe utilitária *Relatorio* no pacote *util*. Por ser utilitária, ela não deve ser instanciável nem herdável. Seu método de classe *emitir(List<Pedido>)* deve retornar o fechamento do dia no formato do Quadro 3: considerando apenas os pedidos entregues, informe, por categoria e na ordem do enum, a quantidade vendida e o faturamento bruto (preço × quantidade) de cada produto; em seguida, o número de pedidos entregues e cancelados e o total reembolsado. Utilize *StringBuilder*, mapas para acumular os totais e *textoFixo()* para o alinhamento.	0,5pt
16	Na classe *Principal*, carregue o cardápio (tratando *CardapioInvalidoException* com uma mensagem amigável) e imprima-o como no Quadro 1. Crie cinco clientes, sendo pelo menos um com aniversário na data de hoje (obtida com *LocalDate.now()*). Para cada cliente, crie um número de pedidos definido por uma constante (sugestão: 8), cada um com de 1 a 4 produtos aleatórios, de 1 a 3 unidades cada, e simule-o até que esteja finalizado. Imprima o histórico completo de um dos pedidos, o hall da fama (*Ranking<Cliente>* com os três melhores, Quadro 4) e o fechamento do dia (*Relatorio.emitir()*).	0,5pt
(!)	Utilizar if, else ou switch para decidir o comportamento com base no estado do pedido, em vez de polimorfismo.	-1,0pt
(!)	Implementar métodos demasiadamente ineficientes do ponto de vista computacional.	-1,0pt
(!)	Não seguir as orientações sobre a criação e entrega do projeto descritas no preâmbulo da prova.	-1,0pt

Prêmio Usain Bolt: o estudante que terminar todas as atividades corretamente primeiro ganha 1pt adicional para usar em outra prova. Você está voando?

***** Boa sorte! *****

Quadro 1 – Exemplo de cardápio impresso e de toString() dos produtos

```
------------------------------ PIZZA ------------------------------
Código  Produto
101     Pizza #101 Margherita (M) - R$ 40,00
102     Pizza #102 Calabresa (G) - R$ 57,20
------------------------------ BEBIDA ------------------------------
201     Bebida #201 Guaraná 2,0L - R$ 13,00
...
```

Quadro 2 – Exemplo de histórico de eventos de um pedido (Pedido.toString())

```
[20/10/2026 14:32:07] #3 Item adicionado | Aberto => Aberto | Total = R$ 40,00
[20/10/2026 14:32:07] #3 Item adicionado | Aberto => Aberto | Total = R$ 53,00
[20/10/2026 14:32:07] #3 REJEITADO: Entregar | Estado = Aberto | Motivo: nao e possivel entregar no estado Aber
[20/10/2026 14:32:07] #3 Pagamento confirmado | Aberto => Pago | Total = R$ 53,00
[20/10/2026 14:32:07] #3 Pedido cancelado | Pago => Cancelado | Total = R$ 53,00
(a mensagem de rejeição é livre, desde que cite o evento e o estado)
```

Quadro 3 – Exemplo de fechamento do dia

```
=================== PIZZATECH - FECHAMENTO ===================
------------------------------ PIZZA --------------------------
Produto              Qtd        Faturamento
Margherita (M)        12          R$ 480,00
Calabresa (G)          7          R$ 400,40
------------------------------ BEBIDA -------------------------
Guaraná 2,0L           9          R$ 117,00
---------------------------------------------------------------
Pedidos entregues: 31
Pedidos cancelados: 9
Total reembolsado: R$ 215,40
---------------------------------------------------------------
```

Quadro 4 – Exemplo de hall da fama

```
==================== HALL DA FAMA ====================
1. Cliente = Maria | Maior pedido = R$ 215,40
2. Cliente = João | Maior pedido = R$ 198,00
3. Cliente = Ana | Maior pedido = R$ 198,00
```

Anexo A – Classe Utilitario (fornecida)

```
package util;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

public class Utilitario {
    // Lê todas as linhas do arquivo. Pode lançar IOException (ex.: arquivo não encontrado).
    public static List<String> carregarLinhas(String caminho) throws IOException {
        return Files.readAllLines(Path.of(caminho));
    }

    // Completa o texto com espaços à direita para alinhar colunas.
    public static String textoFixo(String texto, int tamanho) {
        return String.format("%-" + tamanho + "s", texto);
    }
}
```

Anexo B – Arquivo cardapio.csv (na raiz do projeto)

```
PIZZA;101;Margherita;40.00;M
PIZZA;102;Calabresa;44.00;G
PIZZA;103;Quatro Queijos;52.00;P
PIZZA;104;Portuguesa;48.00;M
BEBIDA;201;Guaraná;6.50;2.0
BEBIDA;202;Suco de Laranja;9.00;0.5
BEBIDA;203;Água;4.00;1.0
SOBREMESA;301;Pudim;12.00;320
SOBREMESA;302;Brownie;14.50;450
```

Para treinar o tratamento de erros, depois que tudo funcionar, teste o seu programa com um arquivo que tenha uma linha com preço “abc”, uma linha com campos faltando e um caminho de arquivo inexistente.